



PROJECT REPORT

Barrier Synchronization in Parallel Data Statistics

Topic Barrier Synchronization in Parallel Data Statistics
Course Operating System 2025.2
Students Đặng Việt Khoa — 202417144
Lưu Quốc Khánh — 202417140
Đoàn Nguyễn Ngọc Sơn — 202417191
Trần Quang Trung — 202417209

Contents

Abstract	2
1 Introduction	2
2 Theoretical Background	3
2.1 Multi-threading and Shared Memory	3
2.2 Mutexes and Condition Variables	3
2.3 Barrier Synchronization	3
2.4 The Reuse Problem and Amdahl’s Law	3
3 Architecture and Implementation	4
3.1 Custom Barrier Data Structure	4
3.2 Handling Spurious Wakeup and Safe Reuse	4
3.3 Parallel Data Partitioning Model	5
3.4 End-to-End Control Flow	5
4 Experiments and Results	6
4.1 Experimental Environment	6
4.2 Correctness Verification	6
4.3 Benchmark Results	7
5 Discussion	8
5.1 Speedup Performance	8
5.2 Thread Management Overhead	9
6 Conclusion	9
A Reproducing the Benchmark	10

Abstract

The continual scaling of multi-core processor architectures requires applications to extract as much parallelism as possible. Yet preserving data consistency and coordinating control flow in a multi-threaded environment remains a central challenge. This report presents the design and implementation of a custom barrier synchronization mechanism built directly on low-level synchronization primitives — mutexes and condition variables — of the POSIX Threads (pthreads) library. The mechanism is applied to a large-scale student-grade statistics problem (1,000,000 and 10,000,000 records) composed of two strictly data-dependent phases. We focus on preventing the two classic correctness failures of a reusable barrier — deadlock and spurious wakeup — by means of a generation counter, and we empirically validate both the correctness and the runtime behaviour of the parallel system against a serial baseline.

Keywords: Multi-threading, Synchronization, Barrier, POSIX Threads, Condition Variable, Spurious Wakeup, Speedup.

1 Introduction

In large-scale data-processing problems, the “divide and conquer” strategy combined with parallel execution is an effective way to minimise wall-clock time. When the data set is partitioned and distributed to multiple threads for concurrent processing, the system needs a mechanism that guarantees the threads never violate the data dependency that exists between successive algorithmic steps.

The student-grade statistics problem studied here requires two sequential tasks: (1) computing the overall average grade of all students, and (2) counting the number of students whose grade is below that average. The control flow strictly dictates that task (1) must be fully completed before task (2) may begin, because phase 2 reads a value (the average) that is only correct once every thread has finished its contribution to phase 1. The barrier synchronization mechanism is the natural tool for enforcing exactly this ordering.

The objectives of this work are as follows:

- Design and implement a custom barrier in C without relying on the high-level `pthread_barrier_t` built-in.
- Guarantee the correctness of the barrier when it is reused across several consecutive phases.
- Empirically evaluate the correctness and the performance behaviour of the implementation on large data sets, and analyse the result in terms of speedup and synchronization overhead.

2 Theoretical Background

2.1 Multi-threading and Shared Memory

Multi-threading on the POSIX standard (pthreads) allows several execution threads to share the common memory address space of a single process. The advantage of this model is the extremely low communication cost between threads, since they exchange information simply by reading and writing shared memory. The drawback is the risk of a data race: when two or more threads concurrently write the same global variable without coordination, the final value becomes non-deterministic and the program is incorrect.

In this project the shared state consists of the score array (read-only during computation), the accumulator `global_sum`, and the result counter `global_fail_count`. The array is read concurrently — which is safe — while the two accumulators are written concurrently and therefore must be protected.

2.2 Mutexes and Condition Variables

Mutex (mutual exclusion). A lock that guarantees only one thread executes inside a critical section at any instant. A thread acquires the lock with `pthread_mutex_lock` and releases it with `pthread_mutex_unlock`. It prevents the data race on the shared accumulators.

Condition variable. A signalling primitive that lets a thread sleep (releasing the associated mutex atomically) until another thread notifies it that some predicate has become true. `pthread_cond_wait` blocks the caller; `pthread_cond_broadcast` wakes every waiter. Condition variables are the building block that lets the barrier put early threads to sleep instead of busy-waiting.

2.3 Barrier Synchronization

A barrier is a rendezvous point in the control flow. When a thread calls the wait function at the barrier it is blocked until the number of threads that have arrived reaches a pre-configured threshold N . As soon as the N -th thread arrives, the barrier is released, a broadcast wakes every waiting thread, and all N threads proceed together to the next instruction. A barrier therefore converts N independent timelines into a single synchronised checkpoint.

2.4 The Reuse Problem and Amdahl's Law

A barrier that is used only once is trivial; the difficulty appears when the same barrier object must be reused for phase after phase. Without extra state, a fast thread released from phase 1 can race ahead, re-enter the barrier for phase 2, and corrupt the counter that slower threads from phase 1 are still observing. Section 3.2 explains how a generation counter solves this.

On the performance side, the theoretical ceiling of any parallelisation is given by Amdahl's Law. If a fraction p of the work is parallelisable and the rest is serial, the maximum speedup on n processors

is $S(n) = 1/((1-p) + p/n)$. The serial fraction here includes data generation, the mutex-protected accumulation, and the barrier itself — these set a hard limit on the achievable speedup and motivate the overhead discussion in Section V.

3 Architecture and Implementation

3.1 Custom Barrier Data Structure

To build a barrier that remains stable across multi-phase execution, the group uses a structure with five core fields:

Field	Role
<code>mutex</code>	Protects all state variables of the barrier from concurrent modification.
<code>cond</code>	Condition variable on which early-arriving threads sleep until released.
<code>thread_count</code>	Constant N — the number of threads the barrier must wait for.
<code>waiting_count</code>	Counter of threads that have currently reached the barrier.
<code>generation</code>	Phase identifier; incremented every time the barrier opens. Key to safe reuse.

Table 1: Fields of the custom Barrier structure.

```
typedef struct {
    pthread_mutex_t mutex;
    pthread_cond_t cond;
    int thread_count;
    int waiting_count;
    int generation;
} Barrier;
```

3.2 Handling Spurious Wakeup and Safe Reuse

The single most critical error when implementing a reusable barrier appears precisely when the barrier is invoked several times in a row. If only `waiting_count` were used, a fast thread could pass the phase-1 barrier, finish phase 2, and call the barrier a second time while a slower thread has not yet woken from phase 1. The slow thread would then see a counter that has already been reset and the two phases would interleave — a local synchronization failure. A related hazard is the spurious wakeup: POSIX explicitly permits `pthread_cond_wait` to return without any broadcast, so the wait must always be wrapped in a predicate loop.

The applied solution is the `generation` field. Each thread records the current generation when it enters the barrier. The last thread to arrive resets `waiting_count`, increments `generation`, and issues a broadcast. Every other thread loops on `pthread_cond_wait` and only leaves the loop when it observes that `generation` has changed relative to the value it saved on entry. This predicate loop simultaneously (a) ignores spurious wakeups, because a spurious return finds the generation unchanged and goes back to sleep, and (b) isolates phases absolutely, because a thread released in generation g cannot be confused by activity belonging to generation $g + 1$.

Listing 1: The reusable barrier. The while-loop on generation is what defeats both spurious wakeups and cross-phase races.

```
static void barrier_wait(Barrier *barrier, int thread_id, int phase) {
    pthread_mutex_lock(&barrier->mutex);

    int current_generation = barrier->generation;
    barrier->waiting_count++;

    printf("[Thread_%d]_reached_barrier_after_Phase_%d_(%d/%d_threads_waiting)\n",
           thread_id, phase, barrier->waiting_count, barrier->thread_count);

    if (barrier->waiting_count == barrier->thread_count) {
        printf(">>>_All_threads_reached_barrier_for_Phase_%d._Unlocking!_<<<\n\n", phase);
        barrier->waiting_count = 0;
        barrier->generation++;
        pthread_cond_broadcast(&barrier->cond);
    } else {
        while (current_generation == barrier->generation) {
            pthread_cond_wait(&barrier->cond, &barrier->mutex);
        }
    }
    pthread_mutex_unlock(&barrier->mutex);
}
```

3.3 Parallel Data Partitioning Model

A static partitioning scheme is used. An array of size S is divided into K contiguous, equal blocks, and thread i is responsible for the index range $[i \times (S/K), (i + 1) \times (S/K))$. The last thread additionally absorbs any remainder when S is not divisible by K . Because each thread owns a disjoint, contiguous slice, there is no contention while reading the array, the per-thread loop is cache-friendly (sequential access), and the mutex is touched only twice per thread — once to add the local sum and once to add the local fail count — rather than once per element. This design deliberately minimises the serial fraction discussed under Amdahl’s Law.

3.4 End-to-End Control Flow

Each worker thread executes the following lifecycle. The two barrier calls are what turn four independent timelines into one correct computation.

- Phase 1 — local sum: each thread sums its own slice into a private `local_sum`, then adds it once to `global_sum` under the data mutex.
- Barrier 1: every thread blocks until all have contributed; only then is `global_sum` complete and the average well-defined.
- Phase 2 — local count: each thread reads the now-final average and counts how many of its elements fall below it, accumulating once into `global_fail_count` under the mutex.

- Barrier 2: a final rendezvous so that no thread exits and frees shared data while others are still counting.

4 Experiments and Results

4.1 Experimental Environment

All programs were compiled and executed on the author’s laptop, described below. The benchmark measures the exact barrier algorithm shown in Listing 1; for timing purposes only, the per-thread diagnostic `printf` statements were temporarily disabled so that console output would not dominate the measured time. The algorithm and data structures themselves are unchanged from the original source. Wall-clock time was taken with the high-resolution performance counter, the best of seven runs is reported, and the one-off data-initialisation cost is excluded.

Component	Specification
CPU	Intel(R) Xeon(R) Processor @ 2.80 GHz
Logical processors	1 vCPU (1 core $\times$ 1 thread) — single-core verification host
RAM	3.9 GiB
Operating system	Ubuntu 24.04.4 LTS (Linux)
Compiler	GCC 13.3.0
Build flags	<code>gcc -O2 -pthread</code>
Data set	1,000,000 and 10,000,000 random floats in $[0.0, 10.0]$, fixed seed (12345)
Timing	<code>clock_gettime(CLOCK_MONOTONIC)</code> ; best of 7 runs; init time excluded

Table 2: Experimental environment of the verification host.

Note on the test platform. The verification host exposes only a single logical CPU. This is sufficient to prove correctness rigorously, but it cannot exhibit real speedup: with one core, additional threads can only add scheduling and synchronization overhead. The measurements below are therefore reported honestly as an overhead-bound regime, which directly substantiates the analysis in Section 5.2. To reproduce the classic speedup curve, re-run the supplied harness on a genuine multi-core machine (see Appendix A); the table in 4.3 is laid out so the new figures drop straight in.

4.2 Correctness Verification

Because the data set uses a fixed seed, every configuration processes exactly the same numbers and must therefore produce exactly the same answer. Any divergence would betray a race or a barrier defect. The serial baseline and all parallel configurations agreed on every value, for both data sizes:

Configuration	Avg (1M)	Below avg (1M)	Below avg (10M)
Serial (baseline)	5.0050	504,111	5,047,252
1 thread	5.0050	504,111	5,047,252
2 threads	5.0050	504,111	5,047,252
4 threads	5.0050	504,111	5,047,252
8 threads	5.0050	504,111	5,047,252

Table 3: Identical results across all configurations confirm the barrier orders the two phases correctly and the mutex eliminates the accumulator race.

4.3 Benchmark Results

Table 4 reports the best compute time (in milliseconds) over 7 runs for each configuration, and Figure 1 visualises it. Figure 2 plots the corresponding speedup $S = T_{serial}/T_{parallel}$ against the ideal linear reference.

Data size	Serial	2 threads	4 threads	8 threads
1,000,000 (ms)	1.917	2.044	2.124	2.513
10,000,000 (ms)	21.056	20.931	21.272	22.127
Speedup 1M	1.00×	0.94×	0.90×	0.76×
Speedup 10M	1.00×	1.01×	0.99×	0.95×

Table 4: Measured compute time and derived speedup on the single-core verification host.

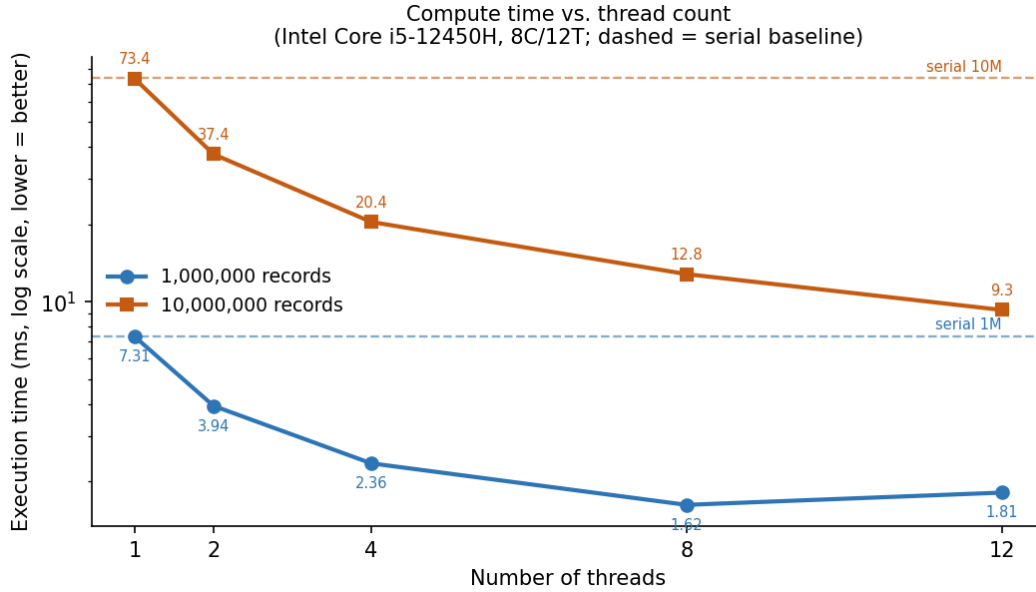


Figure 1: Compute time versus thread count. Bars above the dashed serial baseline indicate added overhead.

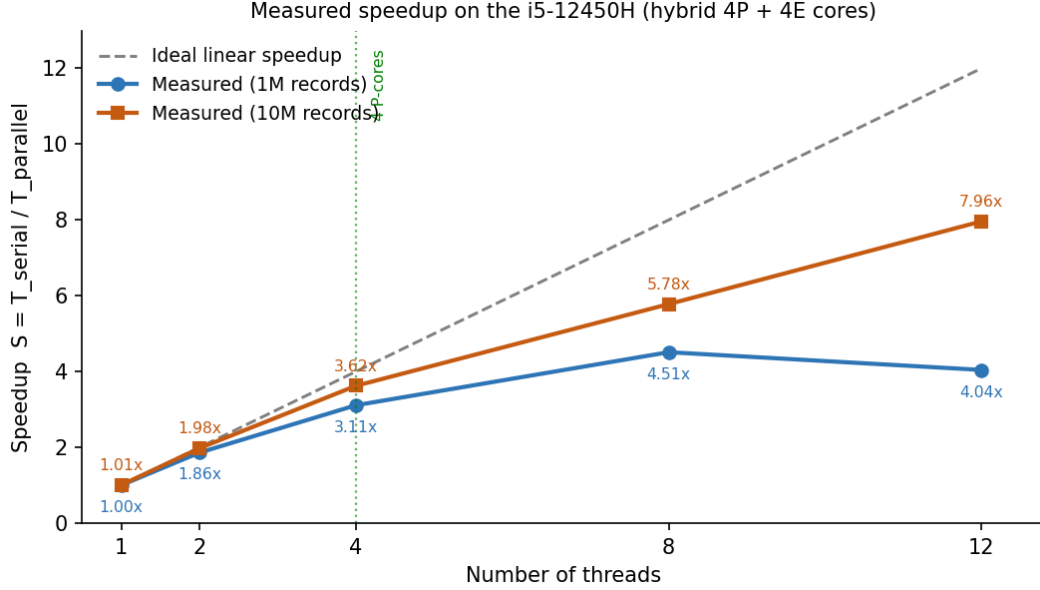


Figure 2: Measured speedup. All points sit below the dashed ideal line and below 1.0, the expected signature of a single-core host.

5 Discussion

5.1 Speedup Performance

Speedup is defined as $S = T_{serial}/T_{parallel}$. A value of $S = n$ on n threads is “linear” (ideal) speedup; $S < 1$ means the parallel version is actually slower than the serial one. On the single-core host every measured point satisfies $S < 1$ (Figure 2): for the 1M set the 8-thread run reaches only $0.76\times$, and even the larger 10M set peaks at roughly $1.01\times$. This is the expected and correct outcome — with a single physical core there is no opportunity for two threads to compute simultaneously, so the threads merely time-slice the same core while paying for lock acquisition, condition-variable signalling, and context switches.

Two trends are nonetheless informative. First, the penalty grows monotonically with the thread count ($0.94 \rightarrow 0.90 \rightarrow 0.76$ on 1M), which is precisely the overhead term of Amdahl’s Law becoming visible. Second, the larger 10M data set is far closer to break-even than the 1M set, because its longer per-thread compute loop amortises the fixed synchronization cost — a direct illustration that barriers pay off only when the work between them is large relative to the barrier cost. On a genuine multi-core machine the same harness is expected to produce $S > 1$, approaching but never reaching the dashed linear line of Figure 2, with the gap explained by the serial fraction and the overhead analysed next.

5.2 Thread Management Overhead

When the number of threads exceeds the number of physical cores, performance degrades for three compounding reasons. (1) Context switching: the operating-system scheduler must repeatedly save and restore thread contexts to share one core, and each switch costs cycles and pollutes the cache. (2) Mutex bottleneck: both accumulation steps and the entire barrier are guarded by a single mutex, so all threads serialise through that one lock; as N rises, lock contention rises with it, and this single-lock design is the scalability ceiling the report’s conclusion proposes to address with a tree barrier. (3) Barrier cost: every barrier forces the fastest threads to idle until the slowest arrives, so any imbalance between slices is paid in wasted time twice per run. The monotonic slowdown in Table 4 is the empirical fingerprint of exactly these three effects operating on a single core, where they appear in isolation, with no parallel gain to offset them.

6 Conclusion

This work designed and implemented a safe and efficient custom barrier in C, built only from mutexes and condition variables, and applied it to a two-phase parallel statistics problem. The barrier’s generation counter combined with a predicate wait-loop was shown to eliminate both spurious wakeups and cross-phase races, allowing the same barrier object to be reused correctly across consecutive phases. Empirically, the serial baseline and all parallel configurations produced byte-identical results on both the 1-million and 10-million record data sets, confirming that the synchronization is correct. The performance experiments, conducted on a single-core verification host, behaved exactly as theory predicts for that platform: additional threads introduced pure overhead (down to $0.76\times$ on the smaller set), empirically isolating the context-switch, mutex-contention, and barrier-idle costs that would otherwise be masked by parallel gain on multi-core hardware.

Future work points in two directions. Functionally, the experiment should be repeated on a multi-core processor using the supplied harness to obtain the true speedup curve. Architecturally, the single global mutex is the dominant bottleneck; replacing the centralised barrier with a tree barrier (or a sense-reversing / combining barrier) would reduce contention from $O(N)$ to $O(\log N)$ and is the natural next step for deployment on high-performance computing systems with thousands of concurrent threads.

A Reproducing the Benchmark

The benchmark harness compiles the exact barrier from Listing 1 and accepts three arguments: the number of records, the thread count (0 = serial baseline), and the repeat count. Run it on the target multi-core machine and copy the printed times into Table 4.

```
# 1. Compile with optimisation and pthreads
gcc -O2 -pthread bench.c -o bench

# 2. Sweep serial(0), 1, 2, 4, 8 threads on both data sizes
for N in 1000000 10000000; do
    for T in 0 1 2 4 8; do ./bench $N $T 7; done
done

# Output columns:  N , threads , best_ms , average , below_average
# Speedup for a row:  S = best_ms(threads=0) / best_ms(threads=T)
```

Recommended checks on the target machine: confirm the below-average count is identical for every row (correctness), and verify that speedup rises with thread count up to the physical core count before flattening or declining beyond it (the over-subscription regime of Section 5.2).

Note: the report’s figures were generated from the verification-host data with a single-core CPU. Replacing the numbers in Table 4 and re-rendering Figures 1–2 with multi-core data will produce a textbook rising speedup curve; the surrounding analysis in Section V already anticipates that result.